```
// ZOARÉ LINE 批發客服機器人（Cloudflare Worker）
// 客人在 LINE 傳文字或照片 → 從《ZOARÉ 日本商品目錄》找商品 → Claude 依客服規則回覆
//
// 需要在 Cloudflare 設定的變數（Settings → Variables and Secrets）：
//   LINE_CHANNEL_SECRET       （Secret）LINE Developers 的 Channel secret
//   LINE_CHANNEL_ACCESS_TOKEN（Secret）LINE Developers 的 Channel access token
//   ANTHROPIC_API_KEY         （Secret）Anthropic Console 的 API Key
//   CATALOG_URL               （Text）catalog.json 的公開網址
//   MODEL                     （Text，可不填）預設 claude-haiku-5-5
// 可選：綁定一個 KV namespace，名稱叫 SESSIONS（記住對話與「真人客服」狀態）

const SHOP = "https://www.lemaiapp.com/m/zoare";
const VIP = "https://www.lemaiapp.com/m/KqNHSrqDUFbRsjgvqKM4qjkFDCI2/p/
qYiJGiQIeyY0yYg5i0dz";
const GENERAL = "https://www.lemaiapp.com/m/KqNHSrqDUFbRsjgvqKM4qjkFDCI2/p/
ETKae1hYHQDfheT9EYci";
const DEFAULT_MODEL = "claude-haiku-5-5";
const HANDOFF_HOURS = 2;

// ---------- 圖文選單：固定回覆（不經過 AI）----------
const askIp = (flag, name, ex) =>
  `{name} 您好，感謝您詢問 ZOARÉ ${name} ${flag}\n請問您想找哪個 IP 或商品呢？\n（例如： ${ex}）\n\n直接告訴我 IP / 商品名稱，或傳商品圖片給我也可以喔！`;
const MENU = {
  "韓國批發": askIp("🇰🇷", "韓國批發", "韓國文具、飾品、服飾、生活小物"),
  "日本批發": askIp("🇯🇵", "日本批發", "三麗鷗、吉伊卡哇、史努比、角落生物；娃娃、吊飾、鑰匙圈"),
  "中國批發": askIp("🇨🇳", "中國批發", "IP 周邊、盲盒、生活雜貨、服飾"),
  "訂單查詢": `{name} 您好，請進入樂賣商城查詢訂單喔！\n🛍️ 樂賣商城： ${SHOP}`,
  "商品查詢": `請給我商品名稱或 JAN 碼供系統查詢喔！\n🛍️ 樂賣商城： ${SHOP}`,
};
const HUMAN_WORDS = ["真人客服", "真人", "找客服", "轉真人"];

// ---------- 客服規則（給 Claude 的系統指令）----------
const RULES = `你是 ZOARÉ 的 LINE 批發客服。ZOARÉ 提供韓國、日本、中國商品批發給網拍、擺攤、娃娃機台主等賣家，不提供個人代購。

【資料來源】
- 日本商品的名稱、價格、規格、最低訂購量、狀態、截止、下單網址，只能用「候選商品」裡的資料。不可以猜、不可以上網查、不可以編造。
- 價格直接用候選商品的「一般」和「VIP」價格，不可以自己計算或打折，也不可以透露成本或加價方式。可以說「VIP 每件比一般價便宜 NT$30」。
```

- 候選商品裡沒有符合的：回答「這個商品目前不在我們的目錄裡，我幫您轉給專人確認喔！」並把 handoff 設為 true。
- 韓國、中國商品沒有目錄，不可以報價：請客人提供商品圖片或連結、型號、數量，告知會由專人報價，handoff 設為 true。

【相同 / 相似商品】
- 從候選商品挑出相同或最相似的商品，編號列出讓客人選。比對順序：JAN / 原廠編號相同 → 名稱相同 → 同 IP / 角色＋同商品種類 (例如同角色不同尺寸) → 同 IP 不同種類或同種類不同角色。
- 客人傳照片：最多列 3 款；打字查詢：最多列 5 款。只有一款完全符合時直接報那一款。
- 狀態是「缺貨/停售」的不要列成選項，可以在最後備註「〇〇 目前缺貨」。
- 沒把握是哪一款時，請客人確認，不要硬報價。
- 每一款格式：
{編號}□ {中文名} ({日文名})
  款式：{款式} (有才寫)
  價格：NT\${一般} | VIP：NT\${VIP}
  最低訂購量：{最低訂購量}
  截止：{截止} (有才寫)
  下單：{下單網址}
- 下單網址是「準備中」時寫「下單連結準備中」，並在最後加一句「需要的話我先幫您登記數量，稍後由專人提供下單連結」。不可自己編網址。
- 列完後說「請回覆編號告訴我您要哪一款，以及需要的數量～」，最後加「※ 日本代購、集單預購」和「💡 VIP 會員每件便宜 NT$30： ${VIP}」。
- 客人回覆編號 (例如「2 號」) 時，從「上一次列給客人的商品」找到那一款，報出完整資料並詢問數量。

【批發規則】
- 非會員：單款 21 件起批。
- 會員 (NT$100 / NT$1,000)：可不同款混批 21 件 (每款 3 件起)，但需依貿易商最低起批數 (目錄的最低訂購量) 為準。不可以說「加入會員後全部一件起」。

【會員】
- 預設只推薦 VIP 會員：NT$1,000 / 月，國際＋國內運費全免、更優惠的批發價、部分商品 VIP 專屬價、新品優先通知。入會： ${VIP}
- 只有客人問 NT$100、問更便宜的方案、或說 VIP 太貴時，才介紹一般會員：NT$100 / 月，只免國際運費。入會： ${GENERAL}

【其他】
- 訂單查詢：請客人進入樂賣商城查詢： ${SHOP}
- 付款方式、運費細節、退換貨、改單、取消、客訴、你不確定的問題：說「我幫您轉給專人確認喔！」，handoff 設為 true。
- 用繁體中文，親切、簡短、專業。客人用日文問可以用日文回。稱呼客人時用 {客人名稱}。
- 不要使用 Markdown (不要用 **、 #、表格)，LINE 不會顯示。

【輸出格式】
直接寫要傳給客人的文字（不要用 JSON、不要用程式碼格式）。
回覆盡量精簡，每款商品只列必要欄位。
最後另起一行加上控制指令（客人看不到）：
#SHOW 要附上商品圖片的候選編號，用逗號分隔，最多 3 個（例如： #SHOW 1,2,3；不需要圖片就寫 #SHOW 0）
需要轉真人客服時，再另起一行寫： #HANDOFF`;

// ---------- 中日文對照與簡繁轉換（搜尋用）----------
const ALIASES = [
  ["酷洛米", "クロミ", "kuromi"], ["美樂蒂", "美乐蒂", "マイメロディ", "マイメロ", "melody"],
  ["凱蒂貓", "凯蒂猫", "kitty", "キティ", "ハローキティ"], ["大耳狗", "喜拿", "シナモロール", "シナモン", "cinnamoroll"],
  ["布丁狗", "ポムポムプリン", "pompompurin"], ["帕恰狗", "ポチャッコ", "pochacco"],
  ["人魚漢頓", "ハンギョドン", "hangyodon"], ["酷企鵝", "ばつ丸", "バッドばつ丸"], ["大眼蛙", "けろっぴ"],
  ["雙星仙子", "キキララ", "kikilala"], ["蛋黃哥", "ぐでたま", "gudetama"], ["三麗鷗", "サンリオ", "sanrio"],
  ["史努比", "スヌーピー", "snoopy"], ["糊塗塌客", "ウッドストック", "woodstock"], ["拉拉熊", "リラックマ", "rilakkuma"],
  ["角落生物", "すみっコぐらし", "すみっこ", "sumikko"], ["吉伊卡哇", "ちいかわ", "chiikawa", "吉伊"],
  ["小八", "ハチワレ", "hachiware"], ["烏薩奇", "乌萨奇", "うさぎ", "兔兔", "usagi"], ["飛鼠", "小桃", "モモンガ"],
  ["塔麻可吉", "たまごっち", "tamagotchi", "電子雞"], ["蠟筆小新", "クレヨンしんちゃん", "小新"],
  ["寶可夢", "ポケモン", "pokemon", "皮卡丘", "ピカチュウ"], ["米飛兔", "ミッフィー", "miffy"],
  ["smiski", "スミスキー"], ["mofusand", "貓福珊迪"], ["小熊維尼", "プー", "pooh"],
  ["娃娃", "玩偶", "絨毛", "ぬいぐるみ", "plush", "蓬鬆", "ほわほわ", "hughug", "fuwakuta", "軟糯", "もちもち"], ["吊飾", "マスコット", "掛飾"],
  ["鑰匙圈", "钥匙圈", "キーホルダー", "キーチェーン"], ["相框", "フォトフレーム"], ["拼圖", "パズル"],
  ["杯", "マグ", "カップ", "タンブラー"], ["便當盒", "ランチボックス", "弁当"], ["包", "バッグ", "ポーチ"],
  ["貼紙", "ステッカー", "シール"], ["盲盒", "扭蛋", "ガチャ", "ブラインド"], ["鏡子", "ミラー"],
  ["毛巾", "タオル"], ["抱枕", "クッション"], ["髮夾", "ヘアクリップ", "クリップ"], ["文件夾", "ファイル"],
  ["電影", "映画"], ["人魚", "人魚の島"],
];
const S2T = { 发: "發", 国: "國", 绒: "絨", 软: "軟", 钥: "鑰", 挂: "掛", 饰: "飾", 头: "頭", 门: "門", 电: "電",
  鱼: "魚", 岛: "島", 秘: "祕", 乐: "樂", 蒂: "蒂", 猫: "貓", 马: "馬", 贴: "貼", 纸: "紙", 镜: "鏡", 包: "包",
  圆: "圓", 盒: "盒", 卡: "卡", 爱: "愛", 夹: "夾", 发夹: "髮夾", 系: "系", 列: "列", 款: "款", 黄: "黃", 蓝: "藍",

```
  红: "紅", 绿: "綠", 粉: "粉", 白: "白", 黑: "黑", 号: "號", 数: "數", 车: "車", 鸟: "鳥", 龙: "龍", 杂: "雜",
  货: "貨", 拼: "拼", 图: "圖", 灯: "燈", 钟: "鐘", 书: "書", 笔: "筆", 记: "記", 袋: "袋", 帽: "帽", 鞋: "鞋",
  萨: "薩", 乌: "烏", 宝: "寶", 梦: "夢", 冲: "衝", 兔: "兔", 熊: "熊", 狗: "狗" };

function norm(s) {
  s = (s || "").normalize("NFKC").toLowerCase();
  let out = "";
  for (const ch of s) out += S2T[ch] || ch;
  return out.replace(/[\s・·、，,。．.()（）\[\]「」『』\/_\-ー～~!！??：:|｜+＋&]/g, "");
}

// ---------- 目錄 ----------
let CATALOG = null;
async function loadCatalog(env) {
  if (CATALOG) return CATALOG;
  const res = await fetch(env.CATALOG_URL, { cf: { cacheTtl: 3600 } });
  if (!res.ok) throw new Error("catalog " + res.status);
  const items = await res.json();
  for (const it of items) it._t = it.t || norm([it.n, it.ja, it.b, it.s, it.code, it.jan].join(" "));
  CATALOG = items;
  return items;
}

const ALIAS_NORM = ALIASES.map((g) => g.map(norm));

function search(items, query, limit = 8) {
  const q = norm(query);
  if (!q) return [];
  const digits = (query.match(/[0-9A-Za-z]{5,}(?:-[0-9]{2})?/g) || []).map((d) => d.toLowerCase());
  if (digits.length) {
    const exact = items.filter((it) => digits.some((d) => d === (it.jan || "").toLowerCase() || d ===
(it.code || "").toLowerCase()));
    if (exact.length) return exact.slice(0, limit);
  }
  // 命中的中日文詞組
  let rest = q;
  const groups = [];
  for (const g of ALIAS_NORM) {
    const hit = g.find((w) => w && q.includes(w));
    if (hit) { groups.push(g); rest = rest.split(hit).join(" "); }
  }
```

```
  const grams = [];
  for (const part of rest.split(" ")) {
    if (part.length === 1) grams.push(part);
    for (let i = 0; i + 1 < part.length; i++) grams.push(part.slice(i, i + 2));
  }
  const scored = [];
  for (const it of items) {
    let sc = 0, groupHits = 0;
    for (const g of groups) if (g.some((w) => it._t.includes(w))) { sc += 10; groupHits++; }
    if (groups.length && !groupHits) continue;
    for (const gr of grams) if (it._t.includes(gr)) sc += gr.length === 1 ? 0.5 : 2;
    if (sc > 0) scored.push([sc - (it.st && it.st.includes("缺貨") ? 3 : 0), it]);
  }
  scored.sort((a, b) => b[0] - a[0]);
  const best = scored×length ? scored[0][0] : 0;
  return scored.filter((x) => x[0] >= best * 0.45).slice(0, limit).map((x) => x[1]);
}

function itemLine(i, it) {
  const f = [`[${i}] ${it.n}`, `日文： ${it.ja || ""}`, `品牌： ${it.b || ""}`];
  if (it.s) f.push(`款式： ${it.s}`);
  f.push(`價格：一般 NT$${it.g} ｜ VIP NT$${it.v}`);
  if (it.spec) f.push(`規格： ${it.spec}`);
  if (it.moq) f.push(`最低訂購量： ${it.moq}`);
  if (it.st) f.push(`狀態： ${it.st}`);
  if (it.ship) f.push(`發售出貨： ${it.ship}`);
  if (it.dl) f.push(`截止： ${it.dl}`);
  if (it.jan) f.push(`JAN： ${it.jan}`);
  else if (it.code) f.push(`原廠編號： ${it.code}`);
  f.push(`下單網址： ${it.url || "準備中"}`);
  return f.join(" ｜ ");
}

// ---------- LINE ----------
async function verifySignature(secret, body, signature) {
  const key = await crypto.subtle.importKey("raw", new TextEncoder().encode(secret), { name:
"HMAC", hash: "SHA-256" }, false, ["sign"]);
  const mac = await crypto.subtle.sign("HMAC", key, new TextEncoder().encode(body));
  const expected = btoa(String.fromCharCode(...new Uint8Array(mac)));
  return expected === signature;
}
```

```
async function lineApi(env, path, payload, host = "api.line.me") {
  return fetch(`https://${host}${path}`, {
    method: payload ? "POST" : "GET",
    headers: { Authorization: `Bearer ${env.LINE_CHANNEL_ACCESS_TOKEN}`, "Content-Type":
"application/json" },
    body: payload ? JSON.stringify(payload) : undefined,
  });
}
async function reply(env, token, messages) {
  const r = await lineApi(env, "/v2/bot/message/reply", { replyToken: token, messages:
messages.slice(0, 5) });
  if (!r.ok) console.log("reply failed", r.status, await r.text());
}
async function displayName(env, userId) {
  try {
    const r = await lineApi(env, `/v2/bot/profile/${userId}`);
    if (r.ok) return (await r.json()).displayName || "老闆";
  } catch (e) {}
  return "老闆";
}
async function imageBase64(env, messageId) {
  const r = await lineApi(env, `/v2/bot/message/${messageId}/content`, null, "api-data.line.me");
  if (!r.ok) throw new Error("image " + r.status);
  const type = (r×headers×get("content-type") || "image/jpeg").split(";")[0];
  const buf = new Uint8Array(await r.arrayBuffer());
  let bin = "";
  for (let i = 0; i < buf.length; i += 0x8000) bin += String.fromCharCode(...buf.subarray(i, i +
0x8000));
  return { type, data: btoa(bin) };
}

// ---------- Claude ----------
async function claude(env, system, messages, maxTokens = 2500) {
  const r = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "x-api-key": env.ANTHROPIC_API_KEY, "anthropic-version": "2023-06-01",
"content-type": "application/json" },
    body: JSON.stringify({ model: env.MODEL || DEFAULT_MODEL, max_tokens: maxTokens,
system, messages }),
  });
  if (!r.ok) throw new Error("claude " + r.status + " " + (await r.text()));
```

```
  const data = await r.json();
  return (data.content || []).filter((c) => c×type === "text").map((c) => c.text).join("");
}
function parseJson(text) {
  // 新格式：純文字 + 最後的 #SHOW / #HANDOFF 控制行
  let t = (text || "").trim();
  // 舊格式相容：如果模型還是回了 JSON，就把 reply 欄位取出來
  if (t.startsWith("{")) {
    try { const j = JSON.parse(t.match(/\{[\s\S]*\}/)[0]); return { reply: j.reply || "", show: j.show || [],
handoff: !!j.handoff }; } catch (e) {}
    const m = t.match(/"reply"\s*:\s*"([\s\S]*)/);
    if (m) t = m[1].replace(/"\s*,\s*"show"[\s\S]*$/, "").replace(/\\n/g, "\n").replace(/\\"/g, '"');
  }
  let show = [], handoff = false;
  const lines = t×split("\n").filter((ln) => {
    const x = ln×trim();
    if (/^#SHOW/i.test(x)) { show = (x××match(/\d+/g) || []).map(Number).filter((n) => n > 0); return
false; }
    if (/^#HANDOFF/i.test(x)) { handoff = true; return false; }
    return true;
  });
  return { reply: lines.join("\n").trim(), show, handoff };
}

// 照片 → 搜尋關鍵字
async function describePhoto(env, img) {
  const out = await claude(env,
    "你是商品辨識助手。看照片判斷商品，只輸出 JSON：{\"query\":\"角色或IP的中文名 日文名 商品
種類 (例：酷洛米 クロミ 娃娃 ぬいぐるみ) 以及照片上看得到的商品名稱\",\"jan\":\"照片上看得到的
JAN 條碼或型號，沒有就空字串\",\"is_product\":true}",
    [{ role: "user", content: [{ type: "image", source: { type: "base64", media_type: img.type, data:
img.data } }, { type: "text", text: "這是什麼商品？ " }] }],
    300);
  try { return JSON.parse(out.match(/\{[\s\S]*\}/)[0]); } catch (e) { return { query: out.slice(0, 80),
jan: "" }; }
}

// ---------- 對話記憶 (有綁 KV 才會啟用) ----------
async function getSession(env, uid) {
  if (!env.SESSIONS) return { history: [], last: [] };
  return (await env.SESSIONS.get("s:" + uid, "json")) || { history: [], last: [] };
```

```
}
async function saveSession(env, uid, s) {
  if (!env.SESSIONS) return;
  s×history = s.history.slice(-8);
  await env.SESSIONS.put("s:" + uid, JSON.stringify(s), { expirationTtl: 86400 });
}
async function inHandoff(env, uid) { return env.SESSIONS ? !!(await env.SESSIONS.get("h:" + uid)) : false; }
async function setHandoff(env, uid) {
  if (env.SESSIONS) await env.SESSIONS.put("h:" + uid, "1", { expirationTtl: HANDOFF_HOURS * 3600 });
}

// ---------- 主流程 ----------
async function handleEvent(env, ev) {
  if (ev×type !== "message" || !ev.replyToken) return;
  const uid = ev.source && ev.source.userId;
  const msg = ev.message;
  const text = msg×type === "text" ? msg.text.trim() : "";

  if (HUMAN_WORDS.some((w) => text === w || text.includes("真人客服"))) {
    await setHandoff(env, uid);
    return reply(env, ev.replyToken, [{ type: "text", text: "好的，已為您通知客服人員，馬上就會有人員與您聯絡囉！ " }]);
  }
  if (uid && (await inHandoff(env, uid))) return; // 真人處理中，機器人不插話
  if (msg×type !== "text" && msg×type !== "image") {
    return reply(env, ev.replyToken, [{ type: "text", text: "請傳文字或商品照片給我，我幫您查詢喔！ " }]);
  }

  const name = uid ? await displayName(env, uid) : "老闆";
  if (MENU[text]) return reply(env, ev.replyToken, [{ type: "text", text: MENU[text].replaceAll("{name}", name) }]);

  if (uid) lineApi(env, "/v2/bot/chat/loading/start", { chatId: uid, loadingSeconds: 20 }).catch(() => {});

  try {
    const items = await loadCatalog(env);
    const session = await getSession(env, uid);
```

```
    let userContent, query = text, img = null;

    if (msg×type === "image") {
      img = await imageBase64(env, msg.id);
      const d = await describePhoto(env, img);
      query = [d.jan, d.query].filter(Boolean).join(" ");
    }
    // 回覆編號（例如「2號」）時沿用上次列出的商品
    const pickedNumber = /^\s×(第\s×)?[1-5一二三四五]\s*(號|个|個|款)?\s*$|[1-5]\s*號/.test(text);
    let cands = pickedNumber && session.last.length ? session.last.map((j) => items[j]).filter(Boolean) : search(items, query);
    if (!cands.length && session.last.length) cands = session.last.map((j) => items[j]).filter(Boolean);

    const candText = cands×length ? cands.map((it, i) => itemLine(i + 1, it)).join("\n") : "（目錄裡沒有找到相關商品）";
    const lastText = session.last.length
      ? session.last.map((j, i) => (items[j] ? `${i + 1}. ${items[j].n}` : "")).join("\n") : "（無）";
    const prompt = `客人名稱： ${name}\n客人${img ? "傳了一張照片（照片辨識結果： " + query + "）" : "說： " + text}\n\n候選商品（依相關度排序）： \n${candText}\n\n上一次列給客人的商品： \n${lastText}`;
    userContent = img
      ? [{ type: "image", source: { type: "base64", media_type: img.type, data: img.data } }, { type: "text", text: prompt }]
      : prompt;

    const messages = [...session.history, { role: "user", content: userContent }];
    const out = parseJson(await claude(env, RULES.replaceAll("{客人名稱}", name), messages));

    const shown = (out×show || []).map((n) => cands[n - 1]).filter((it) => it && it.img).slice(0, 3);
    const lineMsgs = shown.map((it) => ({ type: "image", originalContentUrl: it.img, previewImageUrl: it.img }));
    lineMsgs.push({ type: "text", text: (out.reply || "我幫您轉給專人確認喔！ ").slice(0, 4900) });
    await reply(env, ev.replyToken, lineMsgs);

    session.history.push({ role: "user", content: img ? "（傳了一張照片： " + query + "）" : text });
    session.history.push({ role: "assistant", content: out.reply || "" });
    if (!pickedNumber && cands.length) session.last = cands.map((it) => items.indexOf(it));
    await saveSession(env, uid, session);
    if (out.handoff) await setHandoff(env, uid);
  } catch (e) {
```

```
    console.log("error", e && e.stack || e);
    await reply(env, ev.replyToken, [{ type: "text", text: "系統忙碌中，我先幫您轉給專人，請稍候～" }]);
  }
}

export default {
  async fetch(request, env, ctx) {
    // 去掉金鑰前後不小心多貼的空白或換行
    for (const k of ["LINE_CHANNEL_SECRET", "LINE_CHANNEL_ACCESS_TOKEN", "ANTHROPIC_API_KEY", "CATALOG_URL", "MODEL"])
      if (typeof env[k] === "string") env[k] = env[k].trim();
    const url = new URL(request.url);
    if (request.method === "GET" && url.pathname === "/check") {
      // 只回報設定「有沒有填、長度對不對」，不會顯示金鑰內容
      const s = env.LINE_CHANNEL_SECRET || "", t = env.LINE_CHANNEL_ACCESS_TOKEN || "", a = env.ANTHROPIC_API_KEY || "";
      return Response.json({
        LINE_CHANNEL_SECRET: s ? { length: s.length, looks_ok: /^[0-9a-f]{32}$/i.test(s) } : "missing",
        LINE_CHANNEL_ACCESS_TOKEN: t ? { length: t.length, looks_ok: t.length > 100 } : "missing",
        ANTHROPIC_API_KEY: a ? { length: a.length, looks_ok: a.startsWith("sk-ant-") } : "missing",
        CATALOG_URL: env.CATALOG_URL ? "set" : "missing",
        SESSIONS_KV: env.SESSIONS ? "bound" : "not bound",
      });
    }
    if (request.method !== "POST") return new Response("ZOARÉ LINE bot is running.");
    const body = await request.text();
    const ok = await verifySignature(env.LINE_CHANNEL_SECRET, body, request.headers.get("x-line-signature") || "");
    if (!ok) { console.log("bad signature: check LINE_CHANNEL_SECRET"); return new Response("bad signature", { status: 401 }); }
    const events = (JSON.parse(body).events) || [];
    ctx.waitUntil(Promise.all(events.map((ev) => handleEvent(env, ev))));
    return new Response("ok");
  },
};

// 測試用
export { search, norm, itemLine, RULES, MENU, verifySignature, handleEvent };
```